

An Asynchronous Python Reference Pipeline for High-Throughput OSINT Data Collection: Design, Benchmarks, and Idempotent Deduplication

Ali Murtaza Bhutto

Independent Security Researcher

ORCID: 0009-0007-2787-943X

alibhutto101112@gmail.com

Abstract—Open-source intelligence (OSINT) workflows depend on high-throughput ingestion of public data feeds with strict per-source rate-limit compliance and content-level deduplication across re-runs. We present *osint-pipeline-demo*, a reference asynchronous Python pipeline combining an aiohttp connection pool, per-source token-bucket rate limiting, regex-based field normalisation with SHA-256 content addressing, and batched PostgreSQL writes using `ON CONFLICT DO NOTHING` for idempotent re-runs. We benchmark the system against a controlled local mock with 50 ms per-request latency over 100 requests and measure throughput improvements of $4.93\times$, $9.57\times$, and $18.04\times$ at 5, 10, and 20 concurrent workers respectively over a synchronous baseline. We demonstrate end-to-end against the public NIST NVD CVE API: a re-run of the same source produces zero new inserts, validating the deduplication contract. The pipeline is structured for production use: configuration is externalised, dependencies are pinned for reproducibility, and a 30-test pytest suite exercises the rate limiter, normaliser determinism, retry/back-off behaviour, and live PostgreSQL deduplication.

Index Terms—OSINT, data pipeline, asyncio, aiohttp, PostgreSQL, deduplication, rate limiting, web scraping, cybersecurity, Python concurrency

I. INTRODUCTION

OSINT data collection at scale is bottlenecked by three engineering constraints [1]: (1) per-source rate-limit compliance — public feeds typically permit only tens of requests per minute without authentication, and exceeding the limit results in IP-level temporary blocks; (2) content-level deduplication — re-running an ingestion against an unchanged upstream must not re-insert identical records; and (3) graceful failure recovery — a single transient 5xx must not abort a multi-source run.

Despite the abundance of OSINT tooling in the security literature, a clean reference architecture combining these three constraints is absent. Practitioners typically assemble ad-hoc collectors that lack explicit rate-limit accounting, deduplicate by storing every record and post-filtering, or fail open on transient errors. This paper contributes an open-source reference implementation (*osint-pipeline-demo*) that addresses all three concerns with measurable, reproducible performance characteristics.

Contributions:

- 1) An asynchronous Python pipeline architecture (§III) combining aiohttp connection pooling, per-source token-bucket rate limiting, regex-based field normalisation with SHA-256 content addressing, and batched PostgreSQL writes using `ON CONFLICT DO NOTHING`.
- 2) A reproducible benchmark methodology (§IV) against a controlled local mock that isolates pipeline efficiency from upstream API jitter, with measured speedups between $4.93\times$ and $18.04\times$ versus a synchronous baseline.
- 3) Empirical validation of idempotent deduplication against the public NIST NVD CVE API (§IV-C): 20 records inserted on first run, 0 inserted with 20 duplicates detected on re-run.
- 4) A 30-test pytest suite (§V) including 4 live PostgreSQL integration tests.

II. BACKGROUND

A. OSINT Collection Constraints

OSINT investigations frequently rely on a small number of high-value feeds: vulnerability databases such as the NIST NVD CVE catalogue [2], public corporate registries such as OpenCorporates, code-hosting event streams such as GitHub’s public events feed, and breach-disclosure aggregators. Each feed publishes a rate-limit policy in its terms-of-service that an OSINT tool must respect both for legal compliance [3] and for operational reasons (exceeding the limit results in temporary or permanent blocks).

B. Asynchronous I/O and the GIL

Python’s reference implementation (CPython) serialises native bytecode execution through the Global Interpreter Lock (GIL). For I/O-bound workloads, however, the GIL is released during socket operations, making asynchronous I/O an effective concurrency strategy: a single Python process can hold thousands of in-flight TCP connections via `asyncio` and `aiohttp`. The canonical alternative — threads — pays a context-switch cost that becomes measurable above ≈ 50 concurrent connections.

C. Content-Addressed Deduplication

A SHA-256 digest of canonically-serialised record fields produces a 64-character hex string that is collision-resistant at scale and deterministic across processes (provided the serialisation is canonical: sorted keys, fixed separators) [4]. Using this digest as a UNIQUE constraint in the destination database turns deduplication into a single-statement INSERT with ON CONFLICT DO NOTHING, eliminating the read-then-write race that plagues SELECT-then-INSERT patterns under concurrency.

III. PIPELINE DESIGN

The pipeline is structured as five composable modules connected by a single orchestrator (`src/pipeline.py`).

A. Configuration Layer

`src/config.py` uses `pydantic-settings` to load runtime tunables (worker count, batch size, retry attempts, request timeout, database URL) from environment variables, with defaults suitable for a developer laptop. Source definitions — URL, per-second rate limit, list of named regex extractors — live in `config/sources.yaml`.

B. Rate Limiter

`src/rate_limiter.py` implements an asynchronous token-bucket per source name. The bucket refills at a configurable rate up to a configurable capacity (defaulting to a single-token burst, which gives strict per-second pacing). Two concurrent `acquire()` calls against the same bucket serialise without busy-waiting: the second caller awaits an `asyncio.sleep(wait)` computed from the time-to-next-token.

C. HTTP Collector

`src/collector.py` wraps an `aiohttp.ClientSession` with a TCP connector limited to `workers` concurrent connections. `fetch_all()` spawns one task per source, awaits the rate-limit acquisition, and issues the request inside an `asyncio.Semaphore` that bounds total in-flight requests. Transient 5xx responses and connection errors are retried with exponential backoff: $\text{delay} = \text{base} \cdot 2^{\text{attempt}-1} \pm 25\% \text{ jitter}$. Persistent failure after `retry_max_attempts` retries raises `CollectorError`, which the orchestrator demotes to a logged warning so that one bad source does not abort the run.

D. Normaliser

`src/normaliser.py` applies the source’s named regex extractors to the raw response body, produces a list of records zipped by index, and computes a SHA-256 digest over the canonical JSON serialisation of the field map (sorted keys, no whitespace). Determinism is verified in `tests/test_normaliser.py: test_field_order_does_not_change_hash` confirms that two extractor lists differing only in declaration order produce the same hash for the same input.

E. Storage

`src/storage.py` exposes a single async API (`insert_batch`) that serialises a list of records into a PostgreSQL UNNEST-based bulk insert with ON CONFLICT (`content_hash`) DO NOTHING. The statement returns the number of rows actually inserted, so the caller learns the dedup outcome (rows attempted vs rows persisted) from a single network round-trip.

F. Optional Selenium Pool

`src/selenium_pool.py` provides a lazily-initialised pool of headless Chrome WebDriver instances guarded by an `asyncio.Queue`. Selenium’s blocking `driver.get(url)` is wrapped in `asyncio.to_thread` so the calling coroutine remains non-blocking. The default pipeline does not require Selenium — all three configured sources return JSON over plain HTTP — and the dependency is therefore declared as `[project.optional-dependencies].selenium` rather than a core requirement.

IV. EMPIRICAL EVALUATION

A. Methodology

To isolate pipeline efficiency from upstream API variability, we benchmark against a local `aiohttp` mock server running on the same loopback interface. Each handler sleeps for a configurable latency (default 50 ms) before returning a tiny JSON payload, emulating a bandwidth-cheap but latency-bound upstream.

For each strategy we issue $N=100$ requests and record: wall-clock seconds, throughput (successful requests per second), peak resident set size (RSS) via `getrusage`, and failure count.

Three async strategies are compared against a synchronous baseline (`urllib.request.urlopen` in a single thread):

- `async_5` — `aiohttp` with semaphore-bounded concurrency at 5 workers
- `async_10` — 10 workers
- `async_20` — 20 workers

B. Results

TABLE I
THROUGHPUT BENCHMARK (100 REQUESTS, 50MS UPSTREAM LATENCY)

Strategy	Workers	Wall (s)	Req/s	Peak RSS (KB)	Failures
<code>sync_baseline</code>	1	5.14	19.46	33588	0
<code>async_5</code>	5	1.04	95.89	34100	0
<code>async_10</code>	10	0.54	186.17	34356	0
<code>async_20</code>	20	0.28	351.11	34612	0

The speedup is approximately linear in worker count up to 20 (slope $\approx 0.92\times$ per added worker after accounting for fixed overheads), suggesting the pipeline is not bottlenecked by the GIL or by Python-level work in this regime; further scaling will require either multiple processes or a non-CPython runtime.

```

$ python -m benchmarks.run_benchmarks --requests 100

[sync_baseline] count=100 latency=50.0ms ...
[async_5] count=100 workers=5 ...
[async_10] count=100 workers=10 ...
[async_20] count=100 workers=20 ...

strategy      requests  workers  wall_s  req/s  RSS_kb  failed
-----
sync_baseline    100      1    5.14   19.46   33588    0
async_5          100      5    1.04   95.89   34100    0
async_10         100     10    0.54  186.17   34356    0
async_20         100     20    0.28  351.11   34612    0

Fastest async strategy: async_20 @ 351.1 req/s
Sync baseline:         19.46 req/s
Measured speedup:       18.04x

Results written to benchmarks/results.csv

```

Fig. 1. Real benchmark output (python -m benchmarks.run_benchmarks). Speedup at 20 workers is 18.04x over the synchronous baseline; intermediate worker counts give 4.93x (5) and 9.57x (10).

Peak RSS grows by less than 5% from the baseline at 20 workers (33.6 MB to 34.6 MB), indicating that aiohttp’s connection-pool overhead is negligible compared to the Python interpreter footprint.

C. End-to-End Deduplication Against NIST NVD

To validate the idempotent-rerun contract end-to-end, we ran the pipeline twice against the public NIST NVD CVE API (services.nvd.nist.gov) configured to fetch 20 records. The first run inserted all 20 records; the second run, executed 13 seconds later against the same upstream payload, inserted 0 records and detected 20 duplicates via the content_hash UNIQUE constraint (Fig. 2).

```

$ python -m src.pipeline # real NVD CVE fetch + dedup demo

# First run -- 20 CVEs fetched and inserted
{"ts": "2026-05-02T15:20:30", "level": "INFO", "logger": "osint_pipeline",
 "msg": "fetch ok source=nvd_recent_cves status=200 bytes=42875 attempt=1"}
{"ts": "2026-05-02T15:20:30", "level": "INFO", "logger": "osint_pipeline",
 "msg": "pipeline done duration=0.97s ok=1/1 records=20 inserted=20 dup=0"}

# Second run -- same 20 CVEs, all deduplicated
{"ts": "2026-05-02T15:20:43", "level": "INFO", "logger": "osint_pipeline",
 "msg": "fetch ok source=nvd_recent_cves status=200 bytes=42875 attempt=1"}
{"ts": "2026-05-02T15:20:43", "level": "INFO", "logger": "osint_pipeline",
 "msg": "pipeline done duration=1.15s ok=1/1 records=20 inserted=0 dup=20"}

```

Fig. 2. End-to-end pipeline run against the public NIST NVD CVE API. First run: 20 records inserted. Second run on the same source: 0 inserted, 20 duplicates detected.

V. TEST SUITE

The test suite contains 30 tests across five modules. Tests exercise:

- *Rate limiter* (7 tests): validation, single-source pacing, isolation between sources, concurrent serialisation timing.
- *Normaliser* (7 tests): hash determinism, key-order independence, multi-record extraction, raw-excerpt capping, invalid-UTF8 graceful decode.

- *Collector* (6 tests): retry on 5xx, give-up after exhausted retries, no retry on 4xx, fan-out via aioresponses mock.
- *Config* (6 tests): defaults, env override, sources.yaml round-trip.
- *Storage* (4 tests, integration): real PostgreSQL via docker run, dedup behaviour, mixed-batch idempotency.

```

$ pytest tests/

===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/alim/.../osint-pipeline-demo
configfile: pyproject.toml
plugins: anyio-4.13.0, asyncio-1.3.0
asyncio: mode=auto, asyncio_default_fixture_loop_scope=None
collected 30 items

tests/test_collector.py ..... [ 20]
tests/test_config.py ..... [ 40]
tests/test_normaliser.py ..... [ 63]
tests/test_rate_limiter.py ..... [ 86]
tests/test_storage.py ..... [100]

===== 30 passed in 2.13s =====

```

Fig. 3. Test suite output: 30 tests passing on Python 3.10.12.

Storage integration tests are gated by the PIPELINE_TEST_POSTGRES=1 environment variable so the unit-only subset still runs cleanly on a laptop without a live database.

VI. THREATS TO VALIDITY

Internal validity. The mock-server benchmark deliberately holds upstream behaviour constant; absolute throughput numbers will differ in production. Real public APIs have fewer-than-100 requests per minute rate limits that would dominate wall time. The benchmark measures pipeline efficiency, not end-to-end OSINT collection rate.

External validity. Numbers are reported on a single hardware configuration (Linux 6.8, Python 3.10.12). Other configurations may differ; speedup ratios are likely to be more robust than absolute throughput.

GIL sensitivity. Speedup linearity holds only while the workload is I/O-bound. Adding CPU-heavy normalisation (e.g., machine-learning entity extraction) would re-introduce the GIL as the bottleneck and require multiprocessing.

VII. RELATED WORK

Asynchronous data-pipeline frameworks in Python span a wide design spectrum: lightweight libraries such as Scrapy [5] focus on web-scraping idioms and built-in middleware; production frameworks such as Apache Airflow [6] target scheduled DAGs at scale. Our work occupies a different niche: a minimal, auditable reference implementation showing how the four load-bearing primitives (async fan-out, per-source rate-limit accounting, content-addressed dedup, batched idempotent insert) combine into a sub-1000-LOC pipeline. The legal and ethical constraints applicable to OSINT pipelines are discussed in detail in [3] (DOI 10.5281/zenodo.16924934).

VIII. CONCLUSION

We have presented an open-source reference architecture for high-throughput OSINT data collection in Python and benchmarked it against a controlled local mock, measuring $4.93\times$ to $18.04\times$ speedup over a synchronous baseline at 5 to 20 workers. Idempotent deduplication is validated end-to-end against the public NIST NVD CVE API. The repository is available at <https://github.com/thunderstornX/osint-pipeline-demo>. Future work includes evaluating against Scrapy as a baseline, adding Prometheus metrics for in-production observability, and extending the Selenium pool with anti-detection profile rotation.

REFERENCES

- [1] M. Glassman and M. J. Kang, "Intelligence in the Internet age: The emergence and evolution of Open Source Intelligence (OSINT)," *Computers in Human Behavior*, vol. 28, no. 2, pp. 673–682, 2012.
- [2] National Institute of Standards and Technology, *NVD - National Vulnerability Database API 2.0*, 2024. [Online]. Available: <https://nvd.nist.gov/developers/vulnerabilities>
- [3] A. M. Bhutto, "Legal and Ethical Framework for OSINT Investigations," Zenodo, 2024, doi: 10.5281/zenodo.16924934. [Online]. Available: <https://doi.org/10.5281/zenodo.16924934>
- [4] D. Eastlake and T. Hansen, "US Secure Hash Algorithms (SHA and SHA-based HMAC and HKDF)," RFC 6234, May 2011.
- [5] Scrapy Project, *Scrapy: A fast high-level web crawling and scraping framework*, 2024. [Online]. Available: <https://scrapy.org>
- [6] Apache Software Foundation, *Apache Airflow*, 2024. [Online]. Available: <https://airflow.apache.org>

